408 模拟题

DS

- **数据结构大题**
  - **快慢指针**：链表长度未知时找到中间节点，快指针每次比慢指针多往后一个
  - **链表逆置**：头插法

```
// 将p后的部分逆置(不包括p)
q = p->next;
p->next = NULL;
while(q->next!=NULL) {
    r = q->next; // 先保存要断开的结点后面的
    q->next = p->next; // 插入到头后面，交错进行
    p->next = q;
    q = r;
}

//将p所指的整个链表逆置
q = p->next;
p->next = NULL;
while(q->next!=NULL) {
    r = q->next;
    q->next = p;
    p = q;
    q = r;
}
q->next = p;
```

- 中缀表达式转前缀 中缀表达式：A+B*(C-D)-E/F
  - 法一：用表达式树直接写出表达式
  - 法二：先将序列反转，再求反转后的**后缀表达式**，再将求出的后缀表达式反转
- 表达式树
  - 根据表达式建立 **左右操作数分别在左右节点**，优先级最高先建子树
  - 根据后缀表达式
    - **从左到右**扫描后缀式
    - 遇操作数 → 建节点并压栈

        - 遇运算符 → 弹出两个节点（**右操作数先弹**），建立以该运算符为根的新子树，再压栈
        - 扫描结束后，栈顶节点即为整个表达式树根
    - 根据前缀表达式
        - **从右到左，左操作数先弹**
- 分块查找、索引顺序查找
    - 块内块间均顺序 ASL：(b+1)/2+(s+1)/2，min: 根号n+1
    - 块间折半，块内顺序 ASL
- 双散列法 对于特定的键k，其对应的$h_2(k)$值必须与哈希表大小m互质，满足了这一条件就能保证探测到所有位置
- 根据表达式求后缀表达式
    - 整体加括号，将符号放在括号后面，再去括号
    - 或者用栈
    - 求前缀表达式 符号放括号前
- 十字链表
    - **有向图**
    - 顶点节点存 顶点号、以该顶点为弧头的第一条弧、以该顶点为弧尾的第一条弧
    - 弧节点存 弧尾节点号、弧头节点号、弧尾相同的下一条弧、弧头相同的下一条弧（、权值）
- 邻接多重表
    - **无向图**
    - 顶点节点存 顶点号、第一条边
    - 边节点存 该边连的两个节点、依附两个节点的下一条边（、边权）
    - 依附于同一节点的边串联在同一链表
    - 每个边结点同时在两个链表
- KMP 串的模式匹配
    - **核心**：在模式串 P 第j个位置不匹配跳转到 P 的 next[j] 位置处继续匹配，若不是 P 的第1个不匹配，则主串指针不动
    - 计算 next[j]：next[1]=0, next[2]=1, 其余为**部分匹配值** PM[j-1]+1
      部分匹配值：前缀和后缀最长公共位数
    - ①若用 next 数组确定跳转位置，根据当前模式串中**第 j 个不匹配的 next[j]**，②若直接根据前缀确定右移位数则 j-1-PM[j-1]
    - **优化**：if (P[j]=P[next[j]])，递归 nextval[j]=nextval[next[j]]
    - 先算 PM，其整体加1（序号从1开始，从0开始就不加）
    - **主串不回溯，即序号不减**
    - KMP 右移位数

    - 可直接看
    - j - next[j]
    - j-1 - PM[j-1]
- 快排第二趟排序

- 基数排序基于分配和回收，移动次数与初始状态无关
- 堆的删除**从上至下调整**；**建堆是在原序列上调整**而非重新插入，是从**最后一个非叶节点开始**从上至下调整；而从空的开始插入是从下至上的
  注意**调整过程中比较次数**要**先比较左右孩子结点**，再将父结点与两者中一个比较
- 败者树 **视为完全二叉树**，归并段的数量（路数）为叶结点的数量，非叶节点为败者，根节点为胜者，存的是段号
- 循环队列
    - **rear 指向队尾元素下一个位置**
        - 初始：**front=rear=0**
        - 判空：front=rear
        - 判满：
            - 牺牲一个存储单元：(rear+1)%n=front
            - 增加size：front=rear 与 size
            - 增加tag，删除 tag=0，插入 tag=1：front=rear 与 tag
    - **rear 指向队尾元素**
        - 初始：**front=0**，**rear=n-1**
        - 判空：**(rear+1)%n=front**
        - 判满：
            - 牺牲一个存储单元：(rear+**2**)%n=front
            - 增加size：front=rear + size

    - 增加tag，删除 tag=0，插入 tag=1：front=rear + tag
- ⚠ 在散列表中查找关键字的比较序列是**关键字**，不是其序号
- 数组 a[i] 的地址是(按字节编址、小端或大端方式的)第一个字节的地址，取 a[i] 时是从该地址开始连续取 4 个字节
- B树 第 h 层最少 $2\lceil m/2\rceil^{h-2}$ 个结点，关键字乘 k-1
- "存入（c 语言）**数组**"，**下标-1 !!**
- 哈夫曼编码加权路径长度 WPL(编码总长度)/(各字符**频次之和**)
- B 树插入位置总是在叶结点（最底层终端结点）
  删除操作一定导致叶结点变化
- 森林/树转为二叉树，先序遍历/后序遍历分别是对应二叉树的先序/中序遍历，故根据森林/树的先/后序遍历可确定对应的二叉树
- B 树插入分裂中从中间向上取整或向下取整均可，保证整个操作中用同一种即可
- 前缀编码 可以部分前缀相等，但不能整个是另一个前缀
  010、011√ 110、1100×
- 线索二叉树，先序线索树无法通过链域找**先序前驱**，后序线索树找不到**后继**
- 折半查找树 查找成功最多⌊log2(n+1)⌋，失败最多⌈log2(n+1)⌉，树高
- 二路归并一趟归并调用⌈n/2h⌉次merge，每趟归并段h，整个归并需要⌈log2n⌉趟
  二路归并树高度：初始n个叶结点，倒序/或者高度为趟数+1
  归并排序能用链表实现
- 简单图 不存在重复边、不存在到自己的边
  简单路径 顶点不重复的路径
  简单回路 除第一个顶点和最后一个顶点外，其他顶点不重复

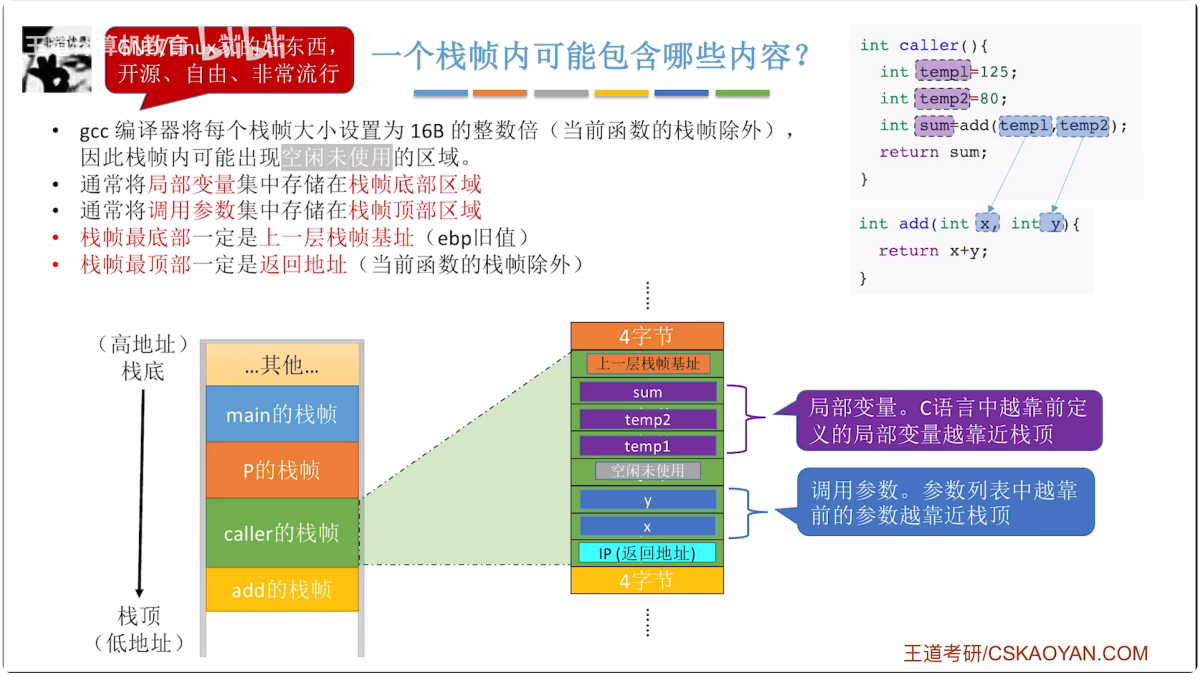
模拟题

- 算法具有的五个特性：有穷、可行、确定、输入、输出
  好的算法达到的指标：正确性、健壮性、可读性、高效率低存储需求
- 前、中、后序遍历**访问叶结点的顺序都一样**
- n 个结点的二叉树形态个数：卡特兰数 (2n)!/n!(n+1)!
- 强连通图：v→w，w→v 之间都有路径，不一定是直接相连，最少的边：一个环路，n
  ⚠ **任意两个点互有路径**（王道第二套.T6）
- B+ 树更适合数据库：支持范围查找
- 快排**每一趟比较次数固定为 n-1**
- 递归遍历二叉树的**左右子树**，如求二叉树最大深度，遍历了所有结点，复杂度是**O(n)**
- 二叉树的某层结点数最多：用选项排除，每层的上一层至少n/2向上取整个，加和不超过总结点数

- B 树的删除 都是落在终端节点，且都是先用拉一个父节点下来，顶一个兄弟结点上去，若兄弟不够借则拉下来的父节点和兄弟结点合并
- 对数的时间复杂度一律以2为底
- 败者树的深度/比较次数
- 折半查找成功/失败的平均查找长度：画出折半查找判定树
- BFS 与 Dijkstra 在一般情况下不是同种算法，但在无权图中它们等价
- ⚠ 注意堆排序是先调整为堆，然后依次输出堆顶元素
- 添加归并段个数：**k-1**-(n-1)%(k-1)
- 完全二叉树判断p、q在同一层次：从0编号，⌊log2(p+1)⌋=⌊log2(q+1)⌋
- 败者树是为了减少元素比较次数
- 并查集
  - ①判断图的连通分量;
  - ②Kruskal 算法求最小生成树;
  - ③判断图中回路;
  - ④验证节点数n=边数+1的树结构，根节点存元素数负值，每次合并边数+1
- 中序线索树从最左结点不断访问右线索或右孩子**不一定**能遍历整棵树，考虑高度为3的满二叉树
- 拉链法的散列表装填因子计算方法同线性

CO

- 函数栈帧



  - 栈帧底部：**上一层栈帧基地址**（ebp旧值）
  - 栈帧顶部：**返回地址**

    - 函数内定义的局部变量：靠近栈帧底部，**越先定义的变量越靠近栈顶** `[ebp-4]`、`[ebp-8]`、`[ebp-12](最先定义的变量)`
    - 调用函数传入的参数：靠近栈帧顶部，**参数列表中越靠左的参数越靠近栈顶**
        - 传递给下一层函数时：`[esp+4]`、`[esp+8]`
        - 访问上一层函数传入的参数 `[ebp+8](第一个参数)`、`[ebp+12](第二个参数)`，ebp 是当前函数栈的基指针
    - 可能有一部分区域用于保存**调用者寄存器**的值
    - 每个栈帧大小是 **16B 整数倍**（当前运行函数除外）
- 调用过程：P 调用 Q
    - 调用者保存寄存器：eax、ecx、edx，P 保存并恢复
    - 被调用者保存寄存器：ebx、esi、edi，Q 保存并恢复
- **调用**

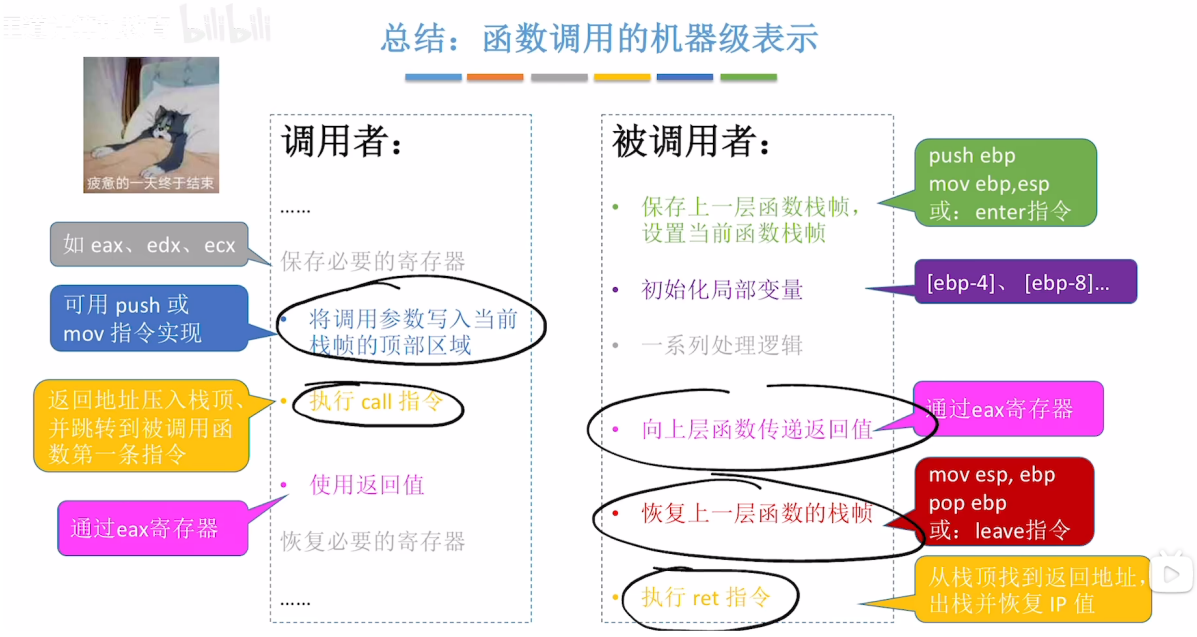
```
(enter 指令)
push ebp // esp-4, ebp -> [esp] 保存上层函数栈帧基址
mov ebp esp  //esp -> ebp 设置当前函数栈帧基址
```

- **返回**

```
(leave 指令)
mov esp ebp
pop ebp //(esp) -> ebp, esp+4

ret // 弹出栈顶并设置为IP跳转
```

- 关注 call 和 ret 指令之前的部分



  - eax 寄存器存返回值
- 注意数组第 i 行的偏移是 i×**列数**
- 页表、页表项的虚拟地址、物理地址总结
  - 用虚拟地址访问页表：页表自映射
  - 页表起始虚拟地址是在该进程的**虚拟地址空间的内核空间中**，该进程的虚拟地址空间起始地址是0
  - **页表项的虚拟/物理地址(装载后)是固定的**，**偏移量**为页表项大小×页号，页号作为索引
  - 注意：若发生缺页异常转换后的物理地址，其页框号是**页表项的内容**
  - 页表所在页对应的页表项**在页表中**，**用偏移算**，其内容是页表的**物理页框号**
- `short si = -32767; unsigned int ui = si;` ui=$2^{32}-2^{15}+1$
  注意是short，符号扩展，扩展后的计算

  由此得到 32 位的 unsigned 的二进制是 1111 1111 1111 1111 1000 0000 0000 0001，其值为 $1+2^{15}+2^{16}+\cdots+2^{31}=2^{32}-2^{15}+1$。
- 单周期 CPU 中的控制器比多周期 CPU 中的更简单
- Cache 索引 是确定主存块映射到Cache的哪个组，直接映射是行号，组相联是组号，全相联没有
  标记是**组内寻找**
- Cache 缺失时间=Cache 命中时间+缺失损失时间
- int 型 `0xff` 高位省略了0，实际上是正数
- ALU主要执行**算术运算**(加、减)和**逻辑运算**(与、或、非、异或、小于)，不包含移位！
- 异常相应过程中CPU完成操作的表述
  - 保存断点和程序状态到内核栈或特定寄存器
  - 关中断

    - 最后跳转到内核中(具体异常)异常处理程序执行
- MMU
    - 访问越界：访问的内存地址超出合法范围（页内偏移大于一页）或已分配的范围
    - **访问越权**：访问超出权限范围的资源
    - MMU 地址转换是将虚拟地址转为物理地址，检测访问越权、访问越界、页面缺失、TLB 缺失，而 Cache 缺失是在**得到物理地址后**检测的
- 区分时间与空间局部性：对于数组相邻元素的访问是空间，时间是要对**同一**元素访问
- Cache命中率
    - 总的访问次数，比如**读后写算两次**
    - 实际缺失次数，注意按行存储的数组按列访问命中率不一定为0！看数组占用的 Cache 块和整个 Cache 的容量，若小于，则访问期间调入的 Cache 块**不会被替换**因而一直留在 Cache 中（原来调入的那块后面会不会因冲突而调出）
- 浮点数的范围大，并不代表它能精确表示所有整数
    - 表示整数时，**相同位数的浮点型精度不如整型**
    - 单精度最大连续**可精确表示**的整数是2^{24}，超过这个值，浮点数的间隔变大，不是每个整数都能表示
- DMA 控制器控制的数据传输通路位于**设备接口和主存**之间
- RISC：精简，指令定长，寄存器多，硬布线控制为主，多组合逻辑，一定流水线
  CSIC：复杂，指令不定长，寄存器少，微程序为主，兼容
- 多路选择器 根据控制信号从**多个输入信号**中选一个直接输出
  地址译码器 将n为地址转为 2^n 个输出信号
  指令字段中的寄存器编号(IR地址部分)送入地址译码器
- IEEE 溢出判断：右规：上溢（127、1023），左规：下溢（**-149、-1074**）
- ⚠ (R1) 是寄存器的**内容而不是间址！**，((R1)) 表示所指的主存单元地址的内容（**一次间址**）
- 页缓冲队列 暂存即将调入主存、等待写回磁盘、被淘汰的页面，减少磁盘I/O，**加快页置换速度**
- SDRAM 同步 DRAM，与 CPU 的数据交换同步于时钟信号，**锁存** CPU 发出的地址和控制信号
- 指令体系结构 ISA
    - 完整定义了**软件和硬件之间的接口**
    - 规定了**指令相关**：指令的格式、寻址方式、操作类型等
    - 规定了**操作数**的类型、寻址方式、数据大小端存放
    - **程序可访问的寄存器**编号、个数，**存储空间**大小、编制方式
    - 指令执行中的**控制方式**：程序计数器、条件码定义
    - 规定了**机器级程序的格式**

- Cache **标记阵列**才会把每组各行（的标记项）**按组排成一行**
  而 r 路组相联的存放方式是**每组 r 行**，故计算组数用 行数/r
- 页表的**有效(装入)位**、替换位(使用位、引用位)、修改位
- 单周期 CPU 每个时钟上升沿开始执行一条新的指令
- 对于寄存器中内容 F000H，补码表示的偏移量 F2H 要**符号扩展** -> FFF2H
- 通常 DMA 接口与主存之间传送单位是**字**，与设备之间传送单位是**字节或位**，宏观是以**块**为单位
- 三种 DMA 传送方式
  - 停止 CPU 访存：DMA 传送一块数据结束再归还总线控制权
  - 周期窃取（周期挪用）：挪用一个**存取周期**，**单字传送**，传送完一个字后立即释放总线，每挪用一个存取周期，都要请求、建立、释放总线控制权
  - 交替访存：CPU 工作周期分为两个时间片，用于 CPU 访存和 DMA 访存，需要 CPU 工作周期比存取周期长，分时控制，不需要请求 建立 释放，主存利用率不高
- DMA请求的响应 可发生在**任意一个机器周期结束时（一个总线事务后）**
- 区分**缺页**和 **TLB 缺失->访问页表**(根据主存地址)
  - 缺页是**地址转换过程中**的异常：是从虚拟地址到主存地址过程中，页表项的有效位为0，说明此时页**还未调入主存**，因此不知道也无法转换得到主存地址
- Cache 标记项包含有效位、修改位、替换算法位、标记位，LRU 算法每行设置一个计数器（LRU 替换位），**位数与每组块数有关**：2路组1位，4路组2位
  算 Cache 总容量注意单位，块大小是 B，而标记阵列是 b
- 程序查询方式 CPU 查询的最长间隔：外设准备一次传输数据的时间（位数/数据率）
  “每隔多少时间进行一次**查询**”，执行输入输出指令的时间<间隔时间
- 时钟中断 处理和时间相关的信息以及决定是否调度程序，能实现进程的切换
- 地址线/数据线复用不能提高总线数据传输率，反而可能降低
- 独立冗余磁盘阵列 RAID 分割交叉存储，并行存取，条带化 将数据分割成多段交替存放，用于提高读写速率
- 低位交叉存储
  - 存取周期 存储器准备数据的时间，如果存储周期不包含取出一个字，**只是准备数据的时间（默认）**，那么一个存储周期准备数据+每隔一个总线周期**启动一次存储器**也就是读取数据，所以流水线时间最后还要加一个总线周期
  - **轮流启动**：每个模块一次读写位数=数据线位数，体数≥m=T/r，**存取周期与总线周期**，读 n 个字：T+(n-1)r
  - **同时启动**：所有模块一次并行读写总位数=数据线位数
- TLB标记
  - 全相联的标记就是**页号**

    - 组相联或直接相联查找时用组号选组，用标记比较
    - 多进程 OS 中，**不同进程虚拟页号可能相同**，需加**地址空间标识符**（ASID）字段，标识不同进程
- 中断**不适合**高速外设，效率很低，**选择型 DMA** 控制器不适用慢速外设，**多路型 DMA** 适合同时为多个慢速外设服务
- 海明码 $n+k \le 2^k-1$，难以区分一位错还是两位错
- 符号扩展 只看**原类型数据类型**，无符号则零扩展，如 unsigned short → int 高位补 0
- RAM 和 ROM 都用MOS管制作存储单元，都是半导体存储器
  闪存 是 ROM，可多次擦写，可替代外存
- 微指令编码
    - 直接编码：一位对应一条微指令，执行速度块，并行性好
    - 字段直接编码：互斥性微命令在同一个段，相容性不在，每个段留一个表示不发出，**二进制**
    - 字段间接编码：一个字段微命令由另一个字段某些微命令解释
- 系统总线的控制总线 传各种**命令**、**反馈**、定时**信号**
- I/O 总线传送的信息
    - **状态信息**、**控制信息**、**中断类型号**由数据线传送，**双向**
    - 控制线和地址线都是**单向**
    - 地址线传IO接口中寄存器地址
    - 控制线传读写控制信号、各种**信号**和**中断请求和响应**
    - 命令字是 CPU 写给 I/O 接口的控制信息，即发送给 I/O 接口的命令；状态字是 I/O 接口反馈给 CPU 的设备当前状态信息，两者都用**数据线**传输
    - IO接口中的数据缓冲寄存器、状态寄存器和命令寄存器的**内容都通过数据线（数据总线）传输**，但寄存器的选择由地址线决定，读写操作由控制线控制
- 上一条指令的ID段被阻塞**导致后一条指令的IF段阻塞**，因为若提前开始IF段，会覆盖指令寄存器、锁存器的内容，导致上一条指令译码结果错误
- 注意**组相联的 LRU**，组内有多行
- CDROM 只读光盘存储器，**串行存取**，**不属于 ROM**
- 数据通路 指令执行过程中数据经过的路径及路径上的部件，包含取指部件，**不包括**控制部件
- 函数调用时，用**栈**保存地址、参数等必要信息；可用循环方式重写递归程序
- 网络中的路由表快速查找：哈希表、前缀树、多级索引表、硬件查找
- B 树和 B+ 树均可用于文件系统（如**磁盘文件索引--已分配的磁盘块**）以及数据库系统
  注意**磁盘空闲块--未分配的磁盘块**的管理使用位图或空闲表、空闲链表
- 主存的 DRAM 芯片构成方式：

    - 顺序方式--高位交叉编址：地址高位为模块号/体号
    - 交叉方式--低位交叉编址：地址低位为模块号/体号，模m编址，如4体则末2位
      数据线位数等于
        - 一个模块每次读写位数：轮流启动，流水线
        - 所有模块一次读写位数：同时启动（一行）并行
- “按字节编址”，若指令23位则需要对齐到字节→24位
- 高级流水线 增加指令级并行：多发射技术（**CPI<1**）与超流水线技术（**CPI=1**）
    - 超标量流水线--动态多发射技术：同一时刻发射多条指令，需要多个功能部件，指令按序发射执行，也可以**动态流水线调度**，**动态分支预测**使指令乱序执行，编译优化
    - 超长指令字--静态多发射技术：编译程序挖掘指令间潜在的并行性，将并行操作的指令组合成具有**多个操作码字段**的超长指令字，多个处理部件
    - 超流水线技术：**提高流水线主频，时钟周期缩短，增加流水线级数，缩短了每个流水线功能段的处理时间**，一个功能部件更频繁的使用，只能按序执行，也要编译优化
- 桥接器 连接不同网络或总线，实现**流量交换**，包括**通信和数据传输**
- I/O 指令实现的数据传送通常发生在**通用寄存器和 I/O 端口之间**，设备通过 I/O 控制器直接与端口交换
- 中断响应阶段的硬件是 CPU 的硬件，所以可以说“CPU在中断响应阶段做的操作”
- 并行总线**效率**比串行总线高，而工作频率无法一直提高
- 区分“主存块号”和“主存地址”映射Cache，**前者没有块内地址部分**
- 汇编程序和编译程序都将源程序解释为机器语言，前者是解释**低级语言**，后者是解释**高级语言**
  汇编语言：机器指令符号表示
- 无符号数比较大小 ZF=0且CF=0，A > B；CF=1，A<B
  有符号数比较大小 OF=SF，ZF=0，A>B；OF!=SF，ZF=0，A<B，各有两种情况
- **Cache 缺失率**
    - 若每组缺失情况相同，只用分析一块内
    - 在程序执行过程中可能多次访问同一个数据，"访问"：**可能读可能写，不要想当然以为只是读，也不要认为缺失一次只用读一次后面就不会访问了**
    - 如循环内 a[k] = a[k] + 32; 一次循环访问了两次 a[k]
- PC 给出的是下一条指令的**地址**，位数与**主存地址空间位数相同**
  IR 直接存指令，位数**同指令位数**
- 硬件描述语言程序 视为高级语言，对**电子系统硬件的行为**尤其**时序行为**进行描述的编程语言
- 要使负数绝对值最大，1尽可能在低位，0尽可能在高位

- 总线定时
  - 同步定时：总线周期
  - 异步定时：握手信号
    - 不互锁方式：不互等
    - 半互锁方式：请求等回答
    - 全互锁方式：请求等回答，回答等请求撤销
  - 半同步定时：wait 信号线采样，**采样由同步时钟控制**，在系统时钟前沿发出，后沿识别，不同速度，可靠性高
  - 分离式定时：上述三种传输周期主从设备完全占据使用权。将总线事务分为**请求**和**应答**，发送完**立即释放总线**，只有单方向信息流，每个设备都是主设备；**仍是同步**，在统一时钟的控制下**将总线周期划分为多个阶段**来提高效率
- **统一编址**下CPU与I/O接口**相连的地址线上**可能传主存地址
  区分："CPU和设备控制接口"或"CPU与I/O接口中的端口"传送的信息不可能是地址
- 用户态下，可以把属于自己**进程地址空间**的内存数据读入寄存器中，但该过程可能由于**缺页中断**而陷入到内核态
- 实现设备无关性 通过设备驱动程序、统一接口、逻辑I/O（系统调用等）
  SPOOLing 有助于设备独立
- 组合逻辑元件（操作元件） 输出只取决于当前的输入，不包含存储信号的记忆单元，不受时钟信号控制，**输入与输出无反馈通路**，信号单向
  时序逻辑元件（状态元件） 输出取决于该时刻与以前，包含，需要**时钟节拍**，如存储器、寄存器
- 输出到总线的部件均需要通过**三态门**和总线连接
- MUX 多路选择器
- 元件控制信号的**取值要看题！**，不能想当然 0/1
- 采用二级页表的系统："访问一级页表"：目录，"访问二级页表"：页表
- 注意**序号问题**，比如地址/磁盘块大小=50，地址%磁盘块大小=12，此时地址应存在第51块，因为地址比前50块单元数大
- 舍入 只可能引起阶码增大或不变
- 混淆了"控制端口"和"控制接口"，某**设备控制接口**实际上就是 **IO 接口**
- 区分"数据缓冲寄存器、控制寄存器、状态寄存器"，"数据端口、控制端口、状态端口"，端口是 CPU 访问寄存器的**地址**
  "数据线、控制线、**地址线**"
- 4M×8位 的 DRAM 芯片地址引脚数据引脚总数 19，**地址引脚一半，行列复用**
- 看清**指令与微指令**
- 引入指令Cache和数据Cache：减少指令流水线资源冲突
  引入多级Cache：降低单级Cache下缺失的时间代价

- 找指令某段的值根据该指令的**机器代码**
- 算 Cache 命中/缺失率时，算 Cache 块能存项多少数据（例如32B的块能存8条4B的指令）有可能缺失率为0
- 访问 TLB 发生缺页时属于**异常**，故中断后需**再访问一次 TLB**：TLB 访问次数+1/再算一次 TLB 访问时间
  区分：Cache 缺失时在处理过程中 CPU 得到块，不需要再次访问
- **访问磁盘块**
  - 顺序移动考虑向前/向后，每次**修改**访问**2次**，读+写
  - 链式存储找到待修改的结点前：只需读，修改的结点读+写
  - 新写入的结点1次
- I/O 接口中状态寄存器和控制寄存器可共用，传送方向相反且访问时间错开；
  对状态端口只能读，控制端口只能写
- 对程序员可见 通用寄存器(含基址和变址)，PSW，PC
  不可见 指令寄存器，暂存寄存器，移位寄存器，ACC
- 基址寻址 采用专门寄存器 BR(对用户可见)，或通用寄存器则要在**指令中增加一段**
- TLB 标记 全相联-**虚页号**，组相联-虚页号**高位部分**，低位部分是**组号**
- 计数单位
  - T=10^{12}
  - P=10^{15}
  - E=10^{18}，1百亿亿
  - Z=10^{21}
- 补码符号相同时，数值位越大，真值越大
- 数据通路包含**异常及中断处理逻辑**
- 设备控制器 即 I/O 接口、I/O 控制器
- “磁盘驱动器”是硬盘设备，“磁盘控制器”才属于 I/O 接口
  “××驱动器”指的是**实际物理设备**，“××控制器/适配器”是指 I/O 控制器
- SSTF 注意每次寻道后都找下一个最短的
- 计算机字长（机器字长） 计算机进行一次**整数运算**所处理的二进制位数，=ALU位数，=通用寄存器位数
  数据线位数 一次**传输**的数据位数
- 指令字长
  - 单字长指令：=机器字长
  - 双字长指令：=2×机器字长，2 个存取周期，访存 2 次
  - 半字长指令：=半个机器字长
- Cache 缺失：硬件
  TLB 缺失：硬件**或软件**

缺页异常：**软件**

- 区：开中断中 CPU 检测到中断请求就响应(√) vs 可屏蔽中断请求信号有效就响应(×)
- 中断服务程序最后一条指令是中断返回，**不需要中断结束信号**
  外设通过中断控制器向 CPU 发送中断结束信号 (×)
  中断控制器：统一管理外设发来的中断请求
- 区分直接寻址 与 立即寻址：直接的地址 or **数**
- 可屏蔽中断请求信号有效 ≠ 开中断
- 浮点寄存器 CPU 中专门用于执行浮点运算与SIMD 向量计算（多个浮点同时运算）的寄存器
- CPU 响应中断有条件：开中断且当前指令执行完
- Cache 访问缺失后将主存块**写入** Cache，**同时设置标记、有效位（、替换位）**
- int 数组元素占 4 字节，按字节编址的系统每隔一个元素，其地址应该**加 4**

模拟题

- C 芯片号+片内地址

  (16) 假定用若干 $8K\times8$ 位的芯片组成一个 $32K\times32$ 位的存储器，存储字长为 32 位，内存按字编址，则地址 $41F0H$ 所在芯片的最大地址是（ ）。

  A. $0000H$　　B. $4FFFH$　　C. $5FFFH$　　D. $7FFFH$

- 指令流水线中 IF、MEM 访存，可能有 TLB/Cache 缺失，而 ID 中的取数、WB 中的写回，都是**针对寄存器堆**的
  只修改寄存器、不访存的指令 MEM 段是空操作
- 一条指令执行过程中，单周期 CPU 的每个控制信号不变，一条指令控制信号同时产生，数据通路每个部件只能使用一次；多周期：改变、多次
- CPU 执行时间 只要 CPU 在运行指令，就算入 CPU 执行时间
  - **包括** I/O 调用、进程调度、**内存管理**、文件系统操作
  - **不包括** I/O 等待时间、程序挂起或阻塞等待时间
- 判断 OF 注意**次高位的进位**是什么，1011+0010 最高位和次高位进位均是 0
- 64M×8位的RAM芯片：每个单元有8位，行数尽量小、行列尽量相同 → **2^13行、列与地址线是13位进行区分**
- 采用LRU替换的组相联映射可能发生数据刚被替换出又被访问：ABCABC...
- 控制冒险 改变PC值的指令
  条件转移指令在 **MEM 段**修改 PC 的值，也称为 WrPC 段，WB 段为空
  插入指令、阻塞/分支预测，延迟损失时间片
- (分支)控制冒险 改变指令执行顺序，转移和返回，中断或异常，有可能产生，不是一定冲突，发生控制冒险的是对应的转移指令等，不是后面被阻塞的！（循环标记处的

指令不是转移就不是控制冒险)

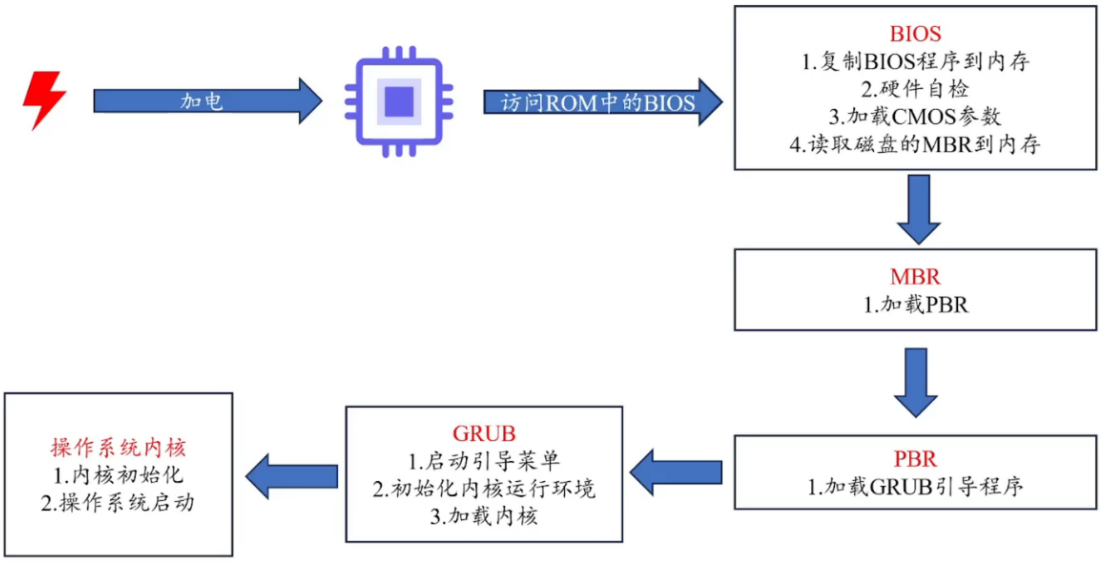
- 提高 Cache 的“关联度”是指：**增加每个 Cache 组中能容纳的块数量**，从而减少冲突，提高命中率
- 适合采用异步传输：速度不同(差异较大)，CPU 和 I/O
- 算两个机器的指令执行速度倍数时，**平均执行时间之比要取倒数**
- 就近舍入：0|1->1，0|0->0
- 寄存器传输指令：寄存器与寄存器之间
- 运算器 包含通用寄存器、**状态寄存器PSW**
  控制器 由 PC、IR、CU(区分指令/数据) 组成，取、分析、执行指令
  运算器和控制器集成到芯片上→CPU
- 一般说寄存器默认是 CPU 中的，Cache 是直接集成在 CPU 中
- 微指令格式
  - 水平型
    - 直接、字段直接、字段间接编码
    - **一位对应一个控制信号**，格式为**操作控制+顺序控制**（判断测试字段+后续地址字段）
      注意对于3个外部条件，互斥则2位，不互斥3位
    - 一条水平型微指令定义并执行多个并行操作的微命令
    - **优点**：微程序短、并发能力强、执行速度快；**缺点**：微指令长
    - 控制存储器容量 微指令字长24位，减去操作控制字段13位(3+4+4+2四个互斥类)与3位判断测试字段，剩8位下地址字段，共有256条微指令，容量为256×24b
  - 垂直型
    - 类似机器指令，设置微操作码，一条垂直型定义并执行一种微命令
    - **优点**：伪指令短、规整、简单、便于编写微程序；
      缺点：微程序长、效率低、速度慢
- 控制存储器中微程序的个数为**机器指令数**+取值间址中断等**公共**微程序数，注意与微指令条数无关（一段微程序若干条微指令）
- 一个时钟周期内可并行传送多次数据
- 闪存翻译层 将来自 CPU 的读写请求翻译成对物理设备的读写控制信号，相当于磁盘控制器
  闪存芯片 相当于磁盘中的机械驱动器
  SSD 以**页为单位读/写**，**块为单位擦除**
- “数据高速缓存”、缓冲存储器 Cache，硬件自动完成读，不能地址指定
- CPU 无法直接访问外存，指令只能使用主存地址
- I1 I2 数据冒险默认**不是** WB 前半段写后半段读

- **状态字(状态信息)是数据**，用数据线传，**状态是信号**，用控制线传
- 多处理器 基于指令流和数据流的数量
  - SISD 一个处理器和一个存储器，一段时间执行一条指令，顺序串行，有的有**多个功能部件**，采用流水线、多模块交叉编址
  - SIMD 一个指令流同时对多个数据流处理，数据级并行（指令序列并发而**数据处理并行**），一个控制部件，多个处理单元，每个单元**有自己的地址寄存器**，对于同一条指令所处理的数据不同。**for 循环**处理数组最高效，**switch case**效率最低变体 向量处理器 直接操作一维数组（向量）指令集的 CPU
  - MISD 不存在
  - MIMD
    - 多计算机系统--**消息传递 MIMD**，各计算机节点存储空间私有，不可通过存取指令，通过消息传递
    - **多处理器系统**（共享存储多处理器 **SMP**）--**共享存储 MIMD**，共享单一地址空间（同一主存），存储指令
    - **线程级及以上并行**
- 硬件多线程 为每个线程提供**单独**的通用寄存器组和PC等，**线程切换**只需**激活选中的寄存器**，省去和存储器交换数据
  - 细粒度多线程 多个线程间轮流交叉执行指令，能在**每个时钟周期**切换线程
  - 粗粒度多线程 仅在当前进程出现较大开销的阻塞时切换，上述两者**指令级并行，线程级不并行**
  - 同时多线程 SMT 同一时钟周期发射不同线程的多条指令，指令级和线程级并行 Intel 处理器中的超线程是 SMT，在**单核/单处理器**中设置**两套线程状态部件**，**共享的是高速缓存和功能部件**
  - 硬件多线程也可用于单核处理器
- 多核处理器 也称片上多处理器，**多个处理单元集成到单个CPU中**，每个核可共享一个 Cache 也可有独立的 Cache。真正意义上并行
- 共享内存多处理器 SMP 共享单一物理地址空间的多处理器
  - 统一存储访问(UMA)多处理器 访存时间相同，通过集成内存控制器的北桥芯片访存
  - 非统一存储访问(NUMA)多处理器 内存控制器集成到 CPU 内部，每个 CPU 独连一部分内存，称为**本地内存**(主存被分配给不同存储器)，CPU 之间通过 **QPI 总线**连接通过 QPI 总线访问其他 CPU 的**远程内存**
- 多核 CPU 的 Cache 一致性还包括各 CPU 的 Cache 之间的一致性，**同一位置的内存数据不同 Cache 的内容应该相同**
- n 个处理机所获得的加速比**小于**一个处理机的n倍
- int转float，可能因为精度而**舍入**

- 地址32位，主存块大小32位，1字32位，全相联，回写，存32K字数据的Cache总容量2048K位，**注意全相联没有行号字段，和直接映射区分**
- 中断字寄存器对程序员不透明，指令缓冲器对程序员透明，移位器、乘法器、先行进位链是ALU的一部分
- IEEE 单精度两操作数阶码之差大于等于25时可直接取阶码较大的数(24时可能加完后发生舍入：01/00+0/11=01/11→10/00)
- n×n位乘法器n次加，n-1次移位
- 中断周期 完成的是对中断的**响应**(寻址)，结束后才会转向执行服务程序

OS

- OS 启动



  - **BIOS**
    - 通电后读取ROM 中 boot 程序找到 BIOS 程序，读到内存
    - 先构建中断向量表，再硬件自检 POST（过程要用到中断）
    - 读 CMOS，加载启动顺序第一位存储设备的引导扇区到内存
    - MBR 位于引导硬盘磁盘块的第一个扇区
  - **MBR**
    - 扫描硬盘分区表，加载硬盘活动分区
    - 加载位于活动分区第一个扇区的分区引导记录 PBR
  - **PBR**
    - 加载 GRUB 引导程序（启动管理器）
  - **GRUB**
    - 启动引导菜单，初始化内核环境，加载内核
- 虚拟机

    - 运行在两类虚拟机管理程序上的OS是 客户OS
    - 第一类虚拟机管理程序（裸金属架构）
        - 每台虚拟机与裸机相同，直接运行在物理硬件上，无须依赖宿主操作系统
        - 唯一一个运行在最高特权级的程序，**在内核空间**
        - 虚拟机作为用户态一个进程运行，上面的OS认为自己在内核态，实则是虚拟内核态，执行内核态指令时陷入虚拟机管理程序，由其代为完成
    - 第二类虚拟机管理程序（寄居架构）
        - 初次启动时类似计算机启动，找驱动器，将OS安装到虚拟磁盘(宿主OS的一个文件)
        - 第二类虚拟机管理程序运行在**用户空间**，系统调用是陷入宿主OS
    - 通过虚拟机技术，可以用一台主机上模拟多种 ISA
- 页式请求管理，进程运行所需最小页框数要考虑指令系统寻址方式，如一级间接寻址-至少需要3个页框：1个指令页 + 1个指针页 + 1个数据页
- 内存映射文件
    - **系统调用**，在进程的虚拟地址空间中分配一段连续的虚拟内存，磁盘文件**直接映射到**进程虚拟地址空间，消除了从页缓存到用户主存的拷贝步骤，仅需一次数据拷贝
    - **将文件当作内存中的字符数组**，进程访问内存的方式读/写文件，而磁盘文件的读/写OS完成，对进程透明
    - 映射时不会实际读入文件内容，只在进程访问映射的虚拟地址时，访问页面时才被每次一页读入物理内存，并将其映射到进程的虚拟地址空间
    - 进程退出或关闭文件映射时写回磁盘
    - 减少数据拷贝、高效内存访问、延迟加载、共享内存实现进程通信
    - 实现了页面到磁盘块的映射
    - 将文件映射到进程的虚拟地址空间，虚拟地址空间最终会通过操作系统的**内存管理单元**（MMU）映射到物理地址空间
- 进程的内存映像
    - 内存映射区域：动态链接库和共享函数库代码
    - 栈：存储函数调用和局部变量
    - 堆：ptr=(char*)malloc(100)，ptr是变量，其**指向的数据才位于堆中**
    - (只读)代码段：.init 程序初始化时调用的函数，.text 用户程序机器代码，**.rodata 只读数据**
    - 读/写数据段：.data 是已初始化的全局变量和静态变量，.bss 是未初始化及初始化为0的全局和静态变量
    - PCB 位于内核区

- 书 P328 I/O 操作流程，进入阻塞态前进程(内核态下)**执行设备驱动程序**，系统调用返回是**最后一步**
  I/O 数据

操作	阶段	说明
将进程 P 插入阻塞队列	系统调用阶段	进程等待输入，进入阻塞
用户在键盘上输入字符	外设阶段	触发硬件中断请求
启动键盘中断处理程序	中断处理	CPU 响应中断，进入ISR
将字符从键盘控制器读入系统缓冲区	中断处理	键盘数据转入系统内存
将进程 P 插入就绪队列	唤醒阶段	进程被唤醒，进入就绪
进程 P 从系统调用返回	调度后恢复	系统调用返回用户态，字符输入完毕

- “建立文件 F1 的硬链接 F3”：让 F3 与 F1 **共享同一个 i-node**
- 若父进程被终止，其子进程未被终止，则成为孤儿进程，被 **init 进程**收养，负责善后
- 内存中**栈有多个**：每个进程/线程都有**独立的用户栈与内核栈**，**只有一个 ebp 寄存器**
- 线程**有专有的存储区**，拥有自己的堆栈，包含在进程地址空间，不共享
- 缓冲 I/O：数据要经过内核缓冲区（中断方式通常采用缓冲IO）
  直接 I/O：DMA，不经过
- 磁头从 0 号磁道 → 399 号，移动 399，不是 400！
- **哲学家就餐**
  - **限制只有 n-1 个人拿筷子**（如通过碗的个数不超过 n-1 个）---**并发最大**
  - 拿碗、拿筷子一气呵成，额外设置一个 mutex 框住
  - 奇偶左右（不推荐）
  - 注意圆桌型的资源用 (i+1)%n
- I/O 控制方式包括硬件特性，和设备驱动紧密相关
- 不同OS提供的系统调用接口**不是统一的**
- 文件分配表 FAT
  - 一个文件系统一张，内存中，显式链接
- 死锁避免 也是**事先预防策略**，都能确保系统不发生死锁，但不会破坏死锁发生的四个必要条件（死锁预防才破坏）
- 设备驱动程序与中断响应程序：设备驱动程序**响应 设备控制器** 的中断请求，**调用**相应的中断响应程序

设备驱动程序 将IO请求转为具体设备的柱面号、磁头号、扇区号
- 进程处理中断过程中不能进行调度，处于内核程序临界区不能调度
- 调度不一定由中断引发，进程创建或结束也能引起调度
- 不管是否支持线程，进程都是**资源分配**的基本单位
- 磁盘分区会使 I/O 具体实现变复杂
- 外部中断和异常处理时都保存**PSW**内容
- 高响应比：（要求服务时间+等待时间）/（**要求服务时间**），主要**作业调度**，满足**短任务优先**，长任务等待时间足够长就比短任务优先级高，无饥饿
- 时间片轮转**不一定**满足短任务优先 **没有考虑时间的长短**
- 多级反馈队列调度算法 也涉及时间片，看到时间片不能只认为是 RR
- 硬链接：基于索引节点，多个指向同一索引节点的指针，速度快
  软链接：符号链接，增加一个保存文件路径的文件，无指针，访问时可能要多次读磁盘
- 命令解释程序（Shell）是操作系统中与用户交互的接口，接收命令 → 解释命令 → 调用系统功能执行命令，在**用户态执行**
- 请求分页系统的内存分配策略没有 固定分配全局置换
  缺页中断 产生后，需要在内存中找到空闲页框并分配给需要访问的页（可能涉及页面置换）
- 中断处理程序 是**最底层**：上下文切换、对处理中断信号源进行测试、读取和修改设备状态
- 操作系统结构
  - 分层法：便于调试验证，易扩充和维护，但不灵活，效率低
  - 模块化：衡量-内聚性和耦合度；**接口通信**，可**直接调用无需消息传递**，可维护和可适应，相互依赖，难以调试验证
  - 宏内核：共享信息，性能优势
  - 微内核：策略与机制相分离，微内核（三个基本功能）+服务器（用户态执行，一组进程），基于客户服务器模式，消息传递机制通信；扩展灵活，可靠安全，与CPU和IO硬件相关代码放内核，服务器与硬件无关；性能问题
    - 进程(线程)管理：进程通信、切换、调度等**功能**，包含优先级队列
    - 低级存储器管理：只配置最基本：地址变换、页表机制，策略、算法的实现放服务器
    - 中断和陷入处理：**捕获**、**响应处理**，具体执行发送到相关服务器
  - 外核：在内核中运行的**程序**，分配资源，减少资源映射层，**无需重映射**，只记录已分配，**让用户直接管理物理资源**，内核仅提供最基本的资源隔离和保护；将多道程序与用户操作系统代码分离；降低统一性，系统更复杂
- 三种装入方式

    - 绝对：编译时
    - 可重定位装入(静态重定位)：装入时转换，装入后不能修改
    - 动态运行时装入(动态重定位)：推迟到执行时产生绝对地址，装入后为相对地址，**重定位寄存器**存装入模块的起始地址。可以不连续，动态申请分配内存，装入后可能换出，便于**共享**
- 三种链接方式
    - 静态链接：运行前，修改相对地址，外部调用符号也都变换成相对地址
    - 装入时动态链接：边装入边链接，便于**修改和更新**，便于**共享**目标模块
    - 运行时：执行中需要时才链接，加快装入过程，节约空间
- 最佳适应算法 空闲分区按容量**递增**排列，外部碎片最多
- 多道程序系统 I/O 设备利用率高
- 系统调用**先将参数压入堆栈，再执行陷入指令**
- “簇号”占2字节，最多2^16个
- 索引表看是否在内存，否则从磁盘中读，**访问文件的磁盘块**还要加1次实际读存有文件内容的磁盘块
- 死锁避免 **没有限制**申请资源的顺序，顺序资源分配法才限制
- 内部异常 有三种，**异常处理结束后**：可恢复故障回到**原断点**，自陷回到**下一条指令**，不可恢复故障或终止异常**终止执行**，即内部异常不一定都回到原断点！
- ⚠区分
    - **SSTF**：每次**最近**
    - **SCAN**：每次向**同一方向**
    - **C-SCAN**：只有**一个方向**
    - 默认LOOK
- 管道 特殊共享文件
    - 固定大小的缓冲区，本质是文件，只有拥有与获得对应文件描述符的进程才能访问
    - 单向，互斥同步，生产者-消费者方式
    - 读数据是一次性操作，读取后释放
    - 子进程继承的是“**文件描述符的副本**”，但**共享同一个管道文件**
- 系统调用时**不会创建新进程**来执行用户进程提出的请求，系统调用是在**当前进程的上下文**中，通过**CPU状态切换**完成用户请求；特定系统调用
- 文件打开 系统打开表和进程打开表；首次打开从外存中复制表项（文件名-索引节点号）到系统打开表，返回索引号（文件描述符）给用户
- 索引节点 文件名和文件描述信息相分离，后者单独形成一个索引节点，文件目录中目录项是**文件名+索引节点号**，FCB 仅是概念上存在
- 进程调度 是OS内核主动选择进程运行，用户**不能**通过系统调用完成进程调度

- “子进程继承父进程的资源”是指**复制**其资源表**而非共享**
- 打开文件表中没有文件名，只有目录项中才有
- **二级页表中页目录项和页表项物理地址**
  - 页框是内存中，物理地址，页框在内存中是**连续**的
  - 页目录项=基地址+**页目录号**×页目录项大小
  - 页表项=**页框号(页目录项内容)×页表(页框)大小**+**页号**×页表项大小
    即对应**页表物理地址**+页表项在页表中的偏移
  - 算物理地址**核心**是**页目录号**(算页目录项的)和**页框号**(页表在内存中的起始)+**页表号**(页表项在页表中的偏移)，**页目录号**和**页表号**通过虚拟地址得出
    但注意要**乘偏移**

模拟题

- 第二类虚拟机管理程序运行在用户态
- 引导扇区的内容是安装操作系统时写进去的
- 引起进程撤销（终止）
  - 正常结束
  - 异常结束（包含算术运算错、I/O故障）
  - 外界干预：操作员或 OS、父进程请求与终止
- **程序顺序执行**具有可再现性，**多道程序并发**则具有不可再现性
- wati 和 signal 可解决一切**互斥**问题，但不能防止死锁
- 设备驱动程序不允许系统调用，本身运行在内核态，基本部分固化在 ROM 中
- DMA 在外设和主存中传数据，同时 CPU 调度执行其他程序，采用 DMA 方式的用户程序被阻塞
- CPU 响应并执行中断时，**不一定**发生了进程切换，**暂停原进程≠切换**，“中断处理程序”**不是某个进程**，**不以任何进程为载体运行，而是执行在特殊的中断上下文中**，它是操作系统内核的一段代码。当前进程无法继续运行时（等IO、缺页等），才阻塞切换进程。中断处理程序结束 *即将返回用户态之前* 由内核决定进程**是否调度和切换**
- VFS
  - 能够把**不同存储介质**（本地磁盘、网络共享、光盘等）和各类文件系统的实现细节**屏蔽**掉
  - 定义了通用文件系统都支持的接口，新文件系统**只要支持这些接口即可安装使用**。抽象了四种对象，每个对象包含数据和函数指针，指向具体文件系统的函数
  - 超级块对象 表示一个已安装(挂载)的特定文件系统，对应磁盘上特定扇区的文件系统超级块，存**已安装的文件系统元信息**
    - 操作函数包括分配、销毁、读写 inode

    - 索引节点对象 表示一个特定的文件，访问文件时在内存创建索引节点对象
        - 操作函数包括创建新索引节点、创建目录项、创建硬链接
    - 目录项对象 是一个路径的组成成分，包含指向关联索引节点的指针、父子目录的指针，其在磁盘上**没有对应的数据结构**，在 VFS 遍历过程中逐个解析为目录项对象
    - 文件对象 表示和进程相关已打开的文件(进程视角)，文件对象和物理文件的关系类似进程和程序的关系
        - 包含该文件的目录项文件、文件系统、文件指针，该文件对象上的一系列操作函数
    - 进程调用 VFS 中的一个函数，该函数调用目标文件系统的相应函数，将请求转换为面向物理设备的指令
- 文件系统挂载 是指把一个独立的文件系统(**设备中**)与操作系统的目录结构（即文件树）关联起来，能够通过目录访问该文件系统上的文件，“设备”是逻辑上的设备，一个磁盘不同分区都能视为不同设备
    - 启动时发现所有设备并安装所有文件系统
    - 安装文件系统的目录称为安装点(挂载点)，一个安装点同时只能挂载一个设备，同一个设备可以有多个安装点，安装完成后 OS 通过访问挂载点即可**透明地**读取、写入该文件系统中的文件，就好像这些文件本来就在本地目录一样-**统一命名空间**
    - 挂载时通过**挂载选项**可以限制对特定文件系统的**读写权限**
    - **跨文件系统协同**：不同类型的文件系统（如 ext4 与 NTFS）可以并存，用户无需关心底层格式，只需通过**统一的路径访问**即可
    - Windows：驱动器字母表示设备和卷，卷 具有常规目录结构，与驱动器号关联，含有**指向已安装文件系统的指针**
    - mount 命令安装文件系统；umount 卸载
- 带权周转时间 作业周转时间/作业实际执行时间
- 死锁预防中破坏请求和保持条件的预先静态分配方法可能会导致饥饿，而仅在运行初期分配并逐步释放的方法则不会
- 固定分区分配 内存空间划分为若干**大小固定**的分区，一个分区只装一道作业，分区大小可相等可不等，**无外部碎片**，不能多个进程共享一个主存区
- SPOOLing 技术输入井和输出井在磁盘上
- 延迟写 是在内存中设置一个**空闲缓冲区队列**，当其他进程**申请到此缓冲区时**才写回磁盘（是被 “空闲缓冲不足” 触发的）
- 单一连续分配的虚拟地址空间=物理
- 多线程模型是线程-线程，多对一模型在**用户态**管理和切换，一个线程访问内核时阻塞，**整个进程都被阻塞**

- 根据算法访问页框缺页次数要算**初始为空时的缺页**
- 设备分配与回收 一个通道连多个控制器，一个控制器连多个设备,设备分配只用创建数据结构
  - 设备控制表 DCT 每个表项是设备各属性，含设备标识符(物理设备名)，设备状态，指向控制器表的指针，指向等待该设备的队列指针；释放设备：将 DCT 中的设备状态设置为空闲
  - 控制器控制表 COCT 含指向对应通道表的指针
  - 通道控制表 CHCT “与通道连接的控制器表首址”表项找到该通道管理的所有控制器信息
  - 系统设备表 SDT 整个系统一张，连接到系统的**所有物理设备**，每个对应一个表目，表目中含有 DCT、**驱动程序入口**
  - 安全分配：请求IO则进入阻塞态，且不保持资源；不安全分配：**请求后继续运行**，只有**请求的设备被占用时才阻塞**，一个进程**同时操作**多个设备
  - 设备分配过程：根据逻辑设备，**从SDT中**→该类设备DCT，若有空闲→该类设备COCT，若有空闲→该类设备CHCT（分配设备--分配控制器--分配通道）
    物理设备名提出IO请求，分配**该设备**的...，不具备设备无关性；以逻辑设备名，则分配**该类设备**的
  - 逻辑设备表 LUT 将逻辑设备名映射为物理设备名，表项为：逻辑设备名、物理设备名、设备驱动程序入口地址，根据逻辑名查找LUT找物理设备名和设驱。两种方式：整个系统一张(适用单用户系统)，每个用户一张
- SPOOLing 由输入进程、输出进程和井管理程序管理
  - 输入/输出进程 负责设备和磁盘井之间的**数据交换**；
  - 井管理程序 负责用户请求的**管理**、输入井/输出井的**空间分配**和对输入输出进程的**调度**。
- 中断处理程序是IO系统最低的一层
- 一级间接寻址至少给一个进程分配3个页框：指令页+间接地址页+操作数页
- 分页存储管理 **没有**逻辑上扩充主存容量，解决的是可以不连续存放
  请求分页存储管理 才扩充
- 分页管理中地址变换过程可能出现的中断：缺页(存在位为0)、越界、**操作越权**(执行的操作和页表中的**保护位**不一致)
- 虚拟设备 是将一个物理设备变成多个逻辑设备，将临界资源→允许同时访问的共享设备
  设备独立性 用统一的逻辑文件名表示多个物理设备
- 进程P执行驱动程序后进入了阻塞态，此时CPU调度其他进程（一般在用户态下运行），IO数据准备好发出中断后，系统进入内核态，中断处理程序运行调用驱动程序将数据从IO端口读入内核缓冲区，P被中断处理程序唤醒，此时可能调度P，也可能

继续返回运行其他进程，当P进程被调度进入运行态后又陷入内核态执行系统调用程序把内核缓冲区的数据复制到用户缓冲区，然后系统调用返回，在用户态运行P

- CPU 利用率：CPU 运行时间/总时间
- 时钟寄存器需要使用**特权指令修改**，对用户不可见
- 逻辑地址→物理地址**发生**在装入时，**该过程**是地址重定

CN

- 差错编码的编码集

  5.1、编码集

  在通信或存储系统中，**编码集**是指用于表示信息的一组码字。每个码字通常是一个固定长度的比特串，通过引入**冗余位**，编码集可以在传输过程中检测或纠正一定数量的错误。

  编码集的关键属性之一是其**最小海明距离**（记作 d）——即任意两个不同码字之间海明距离的最小值。它直接决定了该编码集的 **容错能力**：

  - **检错能力**：最多可以检测 $d-1$ 位错误。
  - **纠错能力**：最多可以纠正 $\left\lfloor \frac{d-1}{2} \right\rfloor$ 位错误。

  最小海明距离越大，意味着码字之间越"分散"，在信道中被干扰后仍然能区分开来，因此检测和纠错能力更强。

  举个实际例子，设有一个编码集，其中包含以下三个 4 位码字：

  ```
  0000, 0110, 1011
  ```

  我们计算这三组码字之间的海明距离：

  - `0000` 与 `0110` 的海明距离为 **2**（第2、3位不同）；
  - `0000` 与 `1011` 的海明距离为 **3**；
  - `0110` 与 `1011` 的海明距离为 **3**。

  因此，该编码集的**最小海明距离**为 $d=2$。

  根据公式，该编码集最多可以：

  - **检测 1 位错误**（$d-1=1$）；
  - **不能纠错**（$\left\lfloor \frac{d-1}{2} \right\rfloor = 0$）。

  海明码检测最多2位错，纠正1位错
- DHCP 发现-提供-请求-确认
  - 发现与请求 源IP是0.0.0.0，提供与确认 源IP是DHCP服务器IP
  - **目的IP都是255.255.255.255**
  - 发现和请求MAC地址是**广播**，提供和确认MAC地址是**单播**
- POP3 可以在一个会话中通过**一条 TCP 连接**获取用户邮箱中的多封邮件
- 注意网络中数据传输的最大吞吐量受限于最小的
- 算信道利用率注意不要算接收方的接收时间！只可能有接收方发送ACK的传输时延！
- 802.11 地址

去往AP	来自AP	地址1	地址2	地址3	地址4
1	0	接收地址=AP地址	发送地址=源地址	目的地址	用于自组网络
0	1	接收地址=目的地址	发送地址=AP地址	源地址	

- 802.11帧 数据部分比以太网长很多
  外部路由器到 AP 之间是802.3帧，AP收到后转换为802.11
- OSPF 分组
  - 问候（HELLO）分组：发现和维持可达性
  - 数据库描述分组：摘要
  - 链路状态请求分组：详细信息
  - 链路状态更新分组：最核心
  - 链路状态确认分组
- BGP
  - 端口号 **179** 半永久的 TCP，发送BGP报文的TCP连接称为 BGP 会话
  - eBGP，外部 BGP 会话：跨越两个 AS
  - iBGP，内部 BGP 会话：同一 AS 内部
  - 转发表表项为到达**网络 X** 的下一跳 <X,R2>
  - 四种报文
    - 打开报文：建立TCP连接之后必须发送，初始化通信，协商参数
    - 保活报文：周期性交换，证实邻站连通性，接收建立BGP连接
    - 更新报文：BGP 核心，撤销(一条或多条)路由，新增(一条)路由
    - 通知(Notification)报文：发送检测的差错
- 数字数据调制为模拟信号
  - 幅移键控 ASK：不同幅度区分01，抗干扰差
  - 频移键控 FSK：不同频率，抗干扰强，容易实现
  - 相移键控 PSK：改变载波相位，绝对调相
  - 差分相移键控 DPSK：检测当前码元和前一个码元相位差，相对
  - 正交幅度调制 QAM：频率不变，AM和PM结合叠加，m个相位n种振幅 log(mn)
- VLAN
  - 基于接口/MAC地址/IP地址
  - 4，最小帧长42，最长1518->1522，前两个字节总是0x8100
  - 交换机根据首部 MAC 地址识别 VLAN

    - 干线链路/汇聚链路 传送 802.1Q帧
    - 收到的是标准以太网帧
- 注意非持久 HTTP/1.0 每次传送完都要**重新建立连接**
- HTTP/1.1 默认持续，分为流水线和非流水线，默认**非流水线**
  HTTP/1.0 可并行建立多个，可没释放完就建立下一个连接（即传输完立刻建立）
- 以太网名称中没有 T 的是同轴电缆（粗缆5和细缆2），带 T 的是双绞线，带 F 的是光纤
- 快重传和快恢复
    - 收到 3 个**冗余** ACK 时
    - 快重传和**立即确认**配套，接收方**收到失序的报文**立刻发送已收到报文的 ACK，发送方收到 3 个冗余 ACK（一共4个）时立即用快恢复重传（一半）而不用等待超时
- 复用 不同应用进程使用同一个传输层协议传送数据；分用 接收方传输层去除首部后交付目的进程
- 网络层的复用 **不同协议的数据**都可以封装成 IP 数据报；网络层分用 接收方网络层去除首部后将数据交付给**相应协议**
- 信道划分介质控制访问
    - 时分复用 TDM：**传输时间**划分为等长的时间片 TDM 帧，m个用户 1/m 带宽
      统计时分复用：异步时分复用，动态分配时隙，可完整带宽
    - 频分复用 FDM：信道总频带划分为子频带，加入隔离频带，多个用户同时，系统效率较高，充分利用带宽，实现容易
    - 波分复用 WDM：光的频分复用，**光纤**中不同波长的光信号，互不干扰，光分用器分解，带宽大
    - 码分复用、码分多址 CDMA：既**共享空间**（信道频率）又**共享时间**，**不会冲突**。发送0时发送序列片反码，无线、移动通信
- MIME 转换、逆转换非 ASCII 码
- 以太网 MAC 协议提供**无连接不可靠**服务，通信质量较差的无线传输中提供**可靠**的服务
- 路由器对分组头**差错校验**：确保相关信息**正确**性
- 看 IP 分组通过以太网是否填充了：首部中的**总长度字段和46比较**
- PPP
    - 两种应用：**用户**连接到某 **ISP** 才能接入互联网，和 ISP 通信时采用；两台网络设备**直连专用线路**
    - 三个组成部分
        - 链路控制协议 LCP：建立、测试、配置**数据链路连接**，**协商选项**
        - 网络控制协议 NCP：每个**不同的网络层协议**对应一个 NCP 来配置

        - 将 IP 数据报**封装**到串行链路的**方法**
    - PPP 帧
        - PPP 面向**字节**，帧的长度整数字节
        - 首尾各有一字节帧定界符 **0x7E**，若该字节出现在信息段中：若使用**异步传输则采用字节填充法**，使用转义字符 **0x7D**；若使用**同步传输则采用零比特填充法**，实现**透明传输**
        - 地址字段（固定0xFF）和协议字段各占1字节
        - 协议字段，表示**信息段运载的什么种类的分组**。0x0021-信息段是 IP 数据报，0xC021-信息段是 LCP 的数据
        - 信息段 0~1500B，无最短帧长限制，点对点
    - 链路静止状态：建立物理层连接后，进入链路建立状态
    - 链路建立状态：协商配置选项，成功后建立 LCP 链路，进入鉴别状态，失败退回静止状态
    - 鉴别状态：成功或无需鉴别则进入网络层协议状态，失败进入链路终止状态
    - 网络层协议状态：采用 NCP 配置网络层，配置完进入链路打开状态，可数据通信
    - 一方发送终止请求并收到确认后，进入链路终止状态，载波停止后，进入静止状态
    - 特点：
        - 建立连接过程中使用确认，但发送数据帧只保证无差错，是**不可靠**服务
        - 仅支持点对点**全双工**，不支持多点
        - 两端可运行不同网络层协议
- TCP 发送窗口题看清是从**连接建立**还是从**连接成功**
- FTP
    - 控制连接：客户连接服务器的 TCP 21 端口
    - 数据连接：
        - 主动 PORT：让**服务器 TCP 20 端口**连**客户随机端口**
        - 被动 PASV：客户连服务器随机端口
- GBN **顺序接收**（关注 T.47）
    - 如何断定已收到？**根据接收方的确认帧**而非接收方帧序号与发送发要求的一致
    - 发送窗口大小根据**数据帧**发送序号的位数或n比特对帧编号：$2^n - 1$
    - 发送窗口在收到确认帧后需要向后移动
- SR
    - 请求重传，逐一确认，无累计确认
    - 数据帧丢失：继续接受缓存其他，等待发送方超时重传

    - 数据帧出错：接收方发送**否定帧 NAK** 发送方收到后立刻重传
    - 发送窗口、接收窗口同步动，大小关系
- DNS 解析 主机中也有缓存，最多8个 UDP 报文，请求+查询
- 滑动窗口算信道利用率中，注意**帧序列比特号与实际发送窗口大小关系**！（GBN、SR）**GBN 最大发送窗口-1**
- NRZ 低0高1，中不变，效率最高，同步问题，双方带时钟线；NRZI 跳0不跳1，中不变，**看起点**，能传时钟信号，增加冗余位实现自同步
- 曼彻斯特 上跳0，下跳1，看中间，中必变；差分曼彻斯特 跳0不跳1，看起点，中必变
- 交换机两种交换方式
    - 直通交换：不校验，直接转发，仅接收6B的目的MAC地址
    - 存储转发
- 调制速率 信号波形变换频率，每秒码元传输个数
- 信道利用率 发送方发送分组到接收到第一个确认分组
    - 连续 ARQ 协议的最大利用率，GBN≥SR，只与发送窗口有关，WT+WR = 2^n
    - 实际传输速率：利用率×带宽；发送数据量/发送周期
- 循环校验码
    - 生成 r 位帧检验序列 FCS：添r个0后除生成多项式（逻辑异或），加上r位余数
    - 检错：补生成多项式最高次个0，校验码是否整除 FCS
- MAC 协议 是一个统称：以太网等共享信道网络中，控制节点如何访问共享信道的机制
  对于以太网，使用的数据链路层协议：IEEE 802.3 协议或CSMA/CD
- IPv6
    - 128 位
    - 首部长度固定 40 字节，扩展首部在有效载荷中，不属于首部
    - 即插即用，自动配置，无需 DHCP
    - 只有源主机才能分片，端到端，不允许中间分片
    - 提出流的概念
    - 与 IPv4 首部字段的不同见书 P179
    - 增加任播：终点是一组，但只交付一个
    - 后64位提取硬件地址，不需要 ARP 协议
- 协议由语法、语义、同步组成
    - 语法 SDU 和 PCI 的格式
    - 语义 需要**做的动作**：需要发出何种控制信息，完成何种动作，做出何种回答
    - 同步（时序） **条件**、顺序

- 分组交换包含虚电路方式和数据报方式
- 虚电路
  - 首部仅在连接建立时使用完整的目的地址，其余只用携带虚电路编号
  - 每个结点维护一张虚电路表，建立过程中确定
  - **路由选择**体现在**连接建立阶段**，一旦建立则路径确定
  - 缺点：某一结点或链路断，经过的所有连接都断
  - 面向连接、可靠、流量控制
  - **有确认的传输**
  - **不必须预分配带宽**，可利用链路上的全部带宽
  - 电路交换 **需要预分配固定带宽**（独占）
- TCP 发送窗口在**每个传输轮次开始时发送**拥塞窗口大小的数据（不考虑接收窗口），每收到一个确认段拥塞窗口+1
  区分**发送**窗口/**拥塞**窗口
  发送窗口=接收窗口<拥塞窗口时，传数据直到发送窗口=0**耗时1个RTT**
- 香农定理不乘2！

模拟题

- 时延带宽积
  - 管道最多能容纳的比特数，第一个比特从发送到接收（链路上比特数量的最大值）
  - 传播时延×传输速率(**链路带宽**)，若给的是信道的**频率带宽**(Hz)需要用两个公式计算传输速率
  - 信噪比：信号功率/噪声功率，注意无单位与单位为db
- 注意 SR 中的发送窗口**不小于**接收窗口，因此不要默认两者相等
- 片偏移 分片在源数据报中的偏移量，8B 为单位，**前一个分片**数据长度/8
  重传 IP 数据报分片时，两次的标识不同（区分）
- 分组的排序(根据序号)在传输层，重组在网络层完成
- TCP 计时器
  - 重传计时器
  - 持续计时器：避免双方死锁等待，收到零窗口通知就启动，超时后发送零窗口探测报文段
  - 保活计时器：避免客户机故障而无效等待
  - 时间等待计时器：MSL
- http 连续连接也得一个一个请求资源

- http 响应报文实体主体部分可能为空：404
- 码元传输速率越大越容易出现接收时判决错误
- ⚠ 子网分配中看清是“可分配的IP数量”和“所包含的IP数量(不减2)”
- TCP 报文段总长度最大不超过 65515B，IP 中总长度字段2B，字节为单位 $2^{16}-20$
- 多播
  - IP 多播地址：224.0.0.0-239.255.255.255
  - 局域网硬件多播
    - IP 多播地址映射为多播 MAC 地址，28->23位，多对一
    - 收到多播数据报的主机在网络层软件过滤，丢弃不是本机要接受的
    - IGMP 网际组管理协议
      多播组成员信息，**只知道本局域网**，不知道互联网，也不知道具体数量.加入多播组时要向多播地址发送 IGMP 报文，本地多播路由器收到后利用多播路由选择协议，将该组成员关系转发给互联网其他多播路由器
      多播路由选择协议：找多播转发树，不会收到重复多播数据报
    - IGMP 报文封装在多播帧
  - 互联网范围+局域网硬件多播
- 二进制指数退避算法第i次重传失败的概率：(0.5)^k, k=min(i,10)
- UDP校验和
  - 不使用，置为全0
  - 计算结果全0，置为全1
  - **结果不可能全1**
- 语义 发出何种控制信息、完成何种动作、做出何种应答，如规定收到请求后执行的操作
- 物理层
  - 不考虑具体的传输媒体(传输介质)，传输介质在物理层下面
  - 所传数据的单位是比特，不用具体考虑比特具体含义
  - 物理层考虑**传输介质**(同轴电缆等)而非传输媒体
- (33) 信道的信号状态数为4,信噪比为30dB时的极限数据传输率为8*kb/s*, 则其带宽约为( )。

  A. 0.8kHz　　B. 2kHz　　C. 0.4kHz　　D. 1kHz

  (B) 注意：①带宽考虑两个定理取最小的 ②香农不是算的码元，不考虑“信号状态数”，只有奈氏准则考虑
- NAV 值不算CTS发送时延和收到ACK后的SIFS，(收到CTS后)SIFS+Data+SIFS+ACK发送时延
- 对于“字节流”中的编号要将b→B
- 没有给子网掩码的路由表按自然分类的默认掩码匹配(如C类→24位)
- 端口号

    - 服务器端：熟知端口号0~1023；登记端口号1024~49151，使用时要登记
    - 客户端：短暂端口号：49152~65535
- 第一次握手的报文(SYN=1，ACK=0)的ack段**无效**
- 香农公式：只要信息传输率低于信道的极限信息传输率，就一定能找到某种方法实现**无差错**传输